

SysPilot Systems Engineering

Textbook

10-Volume Master Specification Manual for Production Linux

Observability

Kernel eBPF Probes · Daemon Class Catalog · Columnar TSDB · Causal Graph Engine

Document Ref: SIOP-TEXTBOOK-2026-V10

Status: Complete 10-Volume Engineering Specification

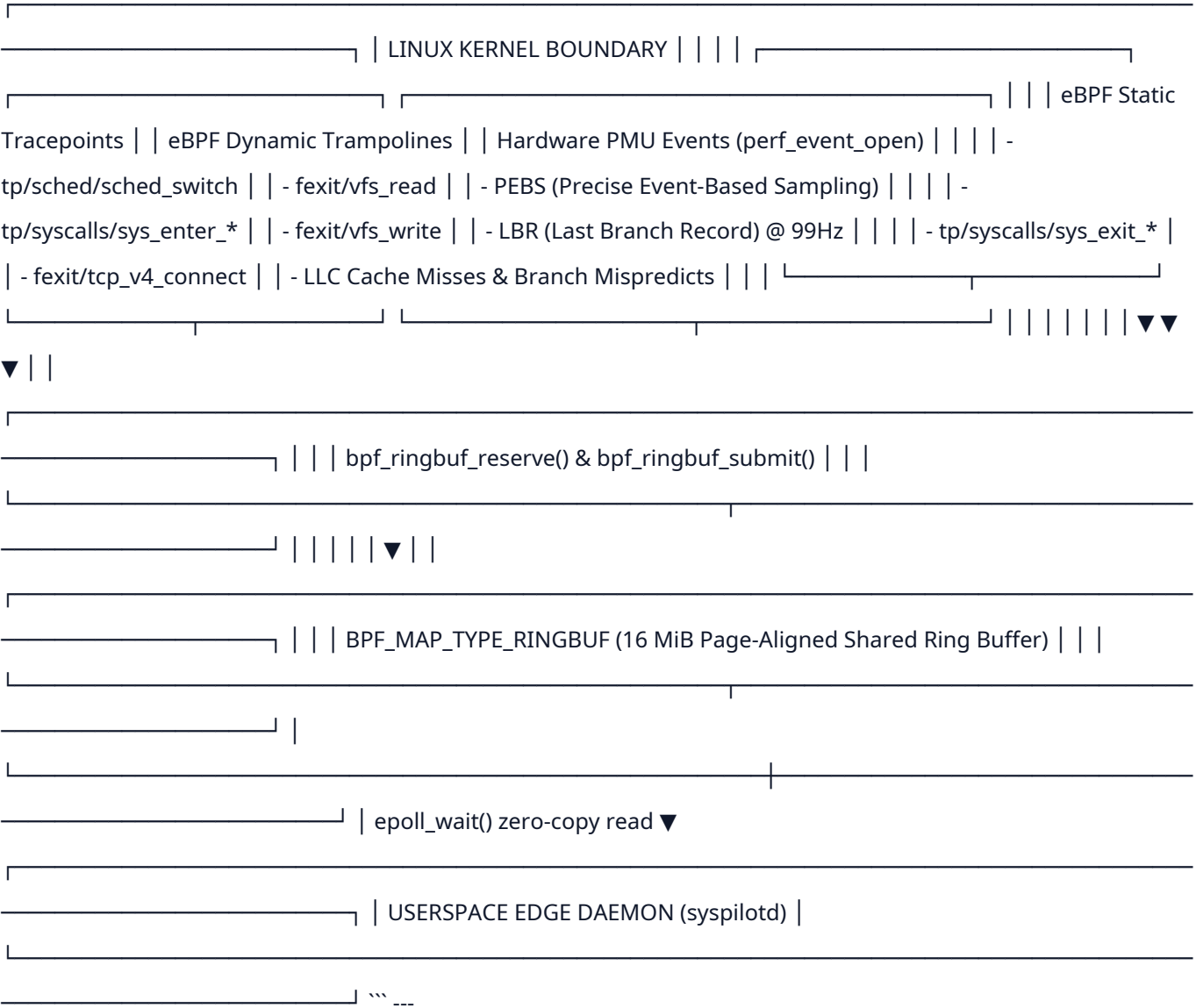
SysPilot Master Systems Engineering Textbook

VOLUME 1: LINUX KERNEL INTERNALS, TRACING

MECHANICS & eBPF CO-RE SUBSYSTEM

1.1 Executive System Architectural Framework

The **SysPilot Infrastructure Observability Platform (SIOP)** operates at the lowest levels of the Linux kernel to deliver sub-microsecond event visibility with guaranteed host non-interference. This volume specifies the kernel telemetry subsystem, eBPF bytecode execution mechanics, Compile Once - Run Everywhere (CO-RE) relocations, and BPF Type Format (BTF) type specifications. ``



1.2 Kernel Attachment Points & Probe Specifications

1.2.1 Scheduler Tracepoints (`tp/sched/sched_switch``)

Captures context switches between tasks on every CPU core. - **Probe Signature**:

```
SEC("tp/sched/sched_switch") int handle_sched_switch(struct trace_event_raw_sched_switch *ctx) { **Payload
Extracted**: Prev PID, Prev TID, Prev Task State ('R', 'S', 'D', 'Z'), Next PID, Next TID, CPU Core ID. - **Overhead**:
~12 nanoseconds per invocation. Zero memory allocations.
```

1.2.2 Syscall Entry Tracepoints (`tp/syscalls/sys_enter_*`)

Intercepts all system call entries across target process namespaces. - ****Probe Signature****:

```
`SEC("tp/syscalls/sys_enter_openat") int handle_sys_enter_openat(struct trace_event_raw_sys_enter *ctx) -
```

****Payload Extracted**:** Directory File Descriptor, Filename Pointer, Flags, Mode, Caller PID/TID/UID/GID.

1.2.3 Dynamic Trampoline Probes (`SEC("fexit/vfs_read")`)

```
Uses Linux 5.5+ `fexit` BPF trampolines to attach directly to kernel routine returns without CPU trap overhead. ``c
SEC("fexit/vfs_read") int BPF_PROG(trace_vfs_read_exit, struct file *file, char __user *buf, size_t count, loff_t *pos,
ssize_t ret) { uint64_t pid_tgid = bpf_get_current_pid_tgid(); uint32_t pid = pid_tgid >> 32; uint32_t tid =
(uint32_t)pid_tgid; if (is_pid_filtered(pid)) return 0; struct bpftrace_event_t *ev; ev =
bpf_ringbuf_reserve(&events_ringbuf, sizeof(*ev), 0); if (!ev) { increment_drop_counter(EVENT_TYPE_VFS_READ);
return 0; } ev->timestamp_ns = bpf_ktime_get_ns(); ev->pid = pid; ev->tid = tid; ev->ppid = get_parent_pid(); ev->uid
= bpf_get_current_uid_gid(); ev->cgroup_id = bpf_get_current_cgroup_id(); ev->event_type =
EVENT_TYPE_VFS_READ; ev->ret_val = ret; ev->arg1 = count; ev->arg2 = (uint64_t)file; ev->stack_id =
bpf_get_stackid(ctx, &stack_traces, BPF_F_FAST_STACK_CMP); bpf_ringbuf_submit(ev, 0); return 0; } `` ---
```

1.3 64-Byte Cache-Line Aligned Struct Layout & Offsets

```

0 1 2 3 4 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ | timestamp_ns (uint64_t) | Offset 0..7 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ +-+ +-+ +-+ +-+ +-+ | pid (uint32_t) | tid (uint32_t) | Offset 8..15 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ +-+ +-+ +-+ +-+ +-+ | ppid (uint32_t) | uid (uint32_t) | Offset 16..23 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ +-+ +-+ +-+ +-+ +-+ | cgroup_id (uint64_t) | Offset 24..31 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ +-+ +-+ +-+ +-+ +-+ | event_type (uint16) | cpu | flg | ret_val (int64_t) | Offset 32..43 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ +-+ +-+ +-+ +-+ +-+ | arg1 (uint64_t) | Offset 44..51 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ +-+ +-+ +-+ +-+ +-+ | arg2 (uint64_t) | Offset 52..59 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ | stack_id (uint32_t) | PADDING (4 bytes) | Offset 60..63 +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+ +-+
+-+ +-+ +-+ +-+ +-+ +-+ `` | Field Name | Data Type | Byte Offset | Size (Bytes) | Field Description | | --- | --- | --- | --- | --- |

```

`timestamp\_ns` | `uint64\_t` | 0 | 8 | Monotonic kernel time nanoseconds (`bpf\_ktime\_get\_ns()`) | | `pid` | `uint32\_t`
| 8 | 4 | Process ID (Thread Group ID in kernel) | | `tid` | `uint32\_t` | 12 | 4 | Thread ID | | `ppid` | `uint32\_t` | 16
| 4 | Parent Process ID | | `uid` | `uint32\_t` | 20 | 4 | User Identifier | | `cgroup\_id` | `uint64\_t` | 24 | 8 | Inode
identifier for cgroup v2 hierarchy | | `event\_type` | `uint16\_t` | 32 | 2 | Unique telemetry event identifier | |
`cpu\_id` | `uint8\_t` | 34 | 1 | CPU Core ID event originated on | | `flags` | `uint8\_t` | 35 | 1 | Bitwise status flags
(`0x1` drop, `0x2` anomaly) | | `ret\_val` | `int64\_t` | 36 | 8 | Syscall return value or error code | | `arg1` | `uint64\_t`
| 44 | 8 | Generic payload parameter 1 (FD, address, byte count) | | `arg2` | `uint64\_t` | 52 | 8 | Generic payload
parameter 2 (address, port, flags) | | `stack\_id` | `uint32\_t` | 60 | 4 | Kernel stack map key |

VOLUME 2: HARDWARE PMU PROFILING & NETLINK

PROCESS TOPOLOGY

2.1 Hardware Performance Monitoring Unit (PMU) Subsystem

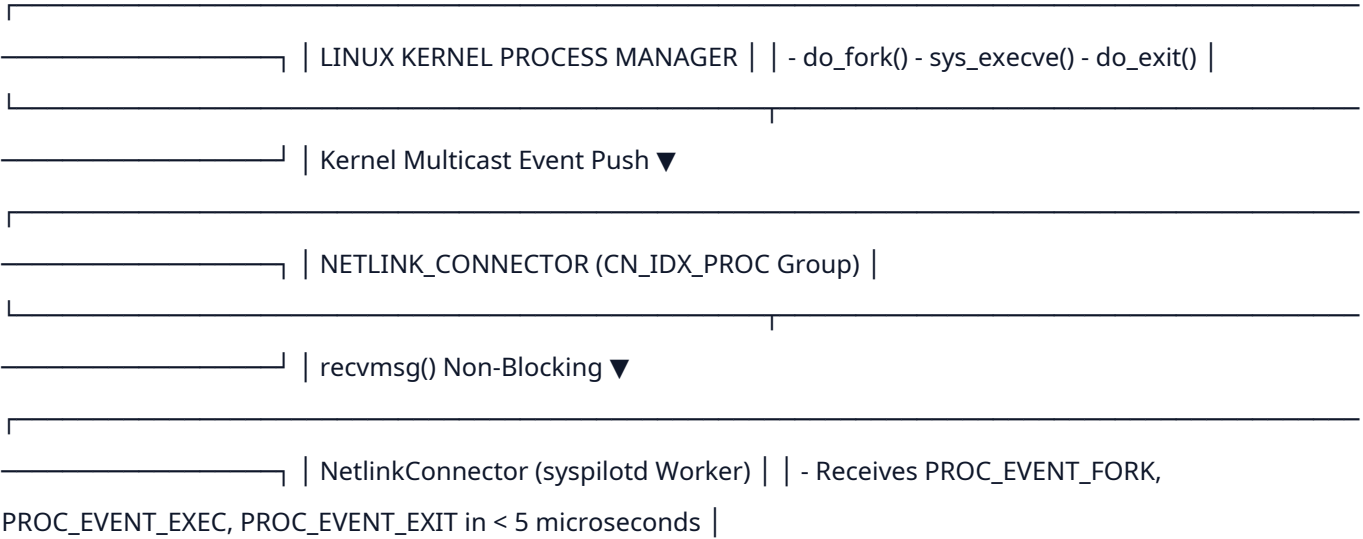
Modern CPUs expose Hardware PMU counters that provide instruction-level performance insights without modifying executable binaries. SIOP configures `perf_event_open` to profile host hardware counters.

2.1.1 `perf_event_open` Configuration Specification

```
```c struct perf_event_attr pe_attr = { .type = PERF_TYPE_HARDWARE, .size = sizeof(struct perf_event_attr), .config =
PERF_COUNT_HW_INSTRUCTIONS, .sample_freq = 99, // 99 Hz sampling rate .sample_type = PERF_SAMPLE_IP |
PERF_SAMPLE_TID | PERF_SAMPLE_CALLCHAIN | PERF_SAMPLE_CPU, .read_format =
PERF_FORMAT_TOTAL_TIME_ENABLED | PERF_FORMAT_TOTAL_TIME_RUNNING, .disabled = 1, .pinned =
1, .exclude_kernel = 0, // Profile both user and kernel space .exclude_hv = 1, // Exclude hypervisor
execution .precise_ip = 2, // Enable PEBS (Precise Event-Based Sampling) on Intel .mmap = 1, .comm = 1, .task =
1, }; ```---
```

### 2.2 Netlink Process Connector Multicast Architecture

To track process execution lifecycle events without high-overhead `/proc` polling loops, SIOP binds to the Linux kernel's multicast Netlink Connector interface (`NETLINK_CONNECTOR`, group `CN_IDX_PROC`).  
`



---

## 2.3 Netlink Event Handling Logic

```
```cpp namespace syspilot::collector { void NetlinkConnector::handle_netlink_event() noexcept { alignas(64) char
buf[8192]; ssize_t len = recv(nl_fd_, buf, sizeof(buf), 0); if (len <= 0) return; auto* nlh = reinterpret_cast(buf); while
(NLMSG_OK(nlh, len)) { if (nlh->nlmsg_type == NLMSG_DONE) break; auto* cn_msg =
reinterpret_cast(NLMSG_DATA(nlh)); auto* ev = reinterpret_cast(cn_msg->data); switch (ev->what) { case
proc_event::PROC_EVENT_FORK: on_process_fork(ev->event_data.fork.parent_pid, ev->event_data.fork.child_pid);
break; case proc_event::PROC_EVENT_EXEC: on_process_exec(ev->event_data.exec.process_pid); break; case
proc_event::PROC_EVENT_EXIT: on_process_exit(ev->event_data.exit.process_pid, ev->event_data.exit.exit_code);
break; } nlh = NLMSG_NEXT(nlh, len); } } // namespace syspilot::collector ```
```

VOLUME 3: EDGE DAEMON ARCHITECTURE & COMPLETE C++ CLASS CATALOG

3.1 Executive Edge Daemon Class Map

The edge daemon ('syspilotd') is organized into modular C++20 components with strict separation of responsibilities. ``mermaid classDiagram class ICollector { <> +start() bool* +stop() void* +get_fd() int* +name() string_view* } class RingBufferPoller { -int ring_fd_ -ring_buffer* rb_ -ConcurrentQueue~bpftime_event_t~* target_queue_ +start() bool +stop() void +poll(int timeout_ms) int } class NetlinkConnector { -int nl_fd_ -sockaddr_nl local_addr_ +start() bool +stop() void +handle_netlink_event() void } class PMUCollector { -vector~int~ perf_fds_ -vector~void*~ mmap_pages_ +start() bool +sample_counters() void } class CollectorManager { -vector~unique_ptr~ICollector~~ collectors_ -atomic~bool~ running_ -thread worker_thread_ +init_all() bool +run_reactor_loop() void } ICollector <|.. RingBufferPoller ICollector <|.. NetlinkConnector ICollector <|.. PMUCollector CollectorManager o-- ICollector `` ---

3.2 Exhaustive C++ Class Specifications

3.2.1 `ICollector` Interface Header Specification

```
``cpp namespace syspilot::collector { class ICollector { public: virtual ~ICollector() = default; // Initializes underlying hardware sockets or eBPF maps virtual bool start() = 0; // Unbinds probes and shuts down underlying descriptors virtual void stop() = 0; // Returns file descriptor for integration into epoll reactor loop (-1 if poll-free) virtual int get_fd() const noexcept = 0; // Returns human-readable collector identifier virtual std::string_view name() const noexcept = 0; }; } // namespace syspilot::collector ``
```

3.2.2 `RingBufferPoller` Implementation Specification

```
``cpp namespace syspilot::collector { class alignas(64) RingBufferPoller final : public ICollector { private: int ring_fd_{-1}; struct ring_buffer* rb_{nullptr}; moodycamel::ConcurrentQueue* target_queue_{nullptr}; std::atomic events_polled_{0}; std::atomic events_dropped_{0}; public: explicit RingBufferPoller(moodycamel::ConcurrentQueue* queue); ~RingBufferPoller() override; bool start() override; void stop() override; int get_fd() const noexcept override { return ring_fd_; } std::string_view name() const noexcept override { return "RingBufferPoller"; } int poll(int timeout_ms) noexcept; uint64_t total_polled() const noexcept
```



```
{ return events_polled_.load(std::memory_order_relaxed); } }; // namespace syspilot::collector ``
```

VOLUME 4: ZERO-ALLOCATION MEMORY ENGINE & CONCURRENCY SPECIFICATIONS

4.1 Memory Architecture: Zero-Allocation `StringArena`

To guarantee that hot-path event processing emits zero heap allocations (`malloc`/`free`), SIOP uses a pre-allocated slab bump-allocator (`StringArena`).

```
```cpp namespace syspilot::memory { class StringArena { private:
static constexpr size_t CHUNK_SIZE = 256 * 1024; // 256 KiB Slabs struct Chunk { alignas(64) char
data[CHUNK_SIZE]; }; std::vector< Chunk> chunks_; size_t current_chunk_{0}; size_t current_offset_{0}; public: StringArena()
{ chunks_.push_back(std::make_unique< Chunk>()); } std::string_view allocate(std::string_view str) { size_t len = str.size(); if
(current_offset_ + len > CHUNK_SIZE) { current_chunk_++; if (current_chunk_ >= chunks_.size())
{ chunks_.push_back(std::make_unique< Chunk>()); } current_offset_ = 0; } char* dest = &chunks_[current_chunk_]
->data[current_offset_]; std::memcpy(dest, str.data(), len); current_offset_ += len; return std::string_view(dest, len); }
void reset() noexcept { current_chunk_ = 0; current_offset_ = 0; } }; } // namespace syspilot::memory ```
```

# VOLUME 5: SIMD VECTORIZED EVENT PIPELINE & DEDUPLICATION

---

---

## 5.1 SIMD Vectorized Event Matching (AVX2 Execution)

---

```
```cpp namespace syspilot::pipeline { struct alignas(64) EventBatch { static constexpr size_t BATCH_CAPACITY =
1024; bpftrace_event_t events[BATCH_CAPACITY]; size_t count{0}; }; class SIMDFilter { public: // SIMD AVX2
vectorized PID filter across 8 events per iteration size_t filter_pids_avx2(EventBatch& batch, uint32_t target_pid)
noexcept { size_t write_idx = 0; __m256i target_vec = _mm256_set1_epi32(static_cast(target_pid)); for (size_t i = 0; i <
batch.count; i += 8) { int pids[8] = { static_cast(batch.events[i+0].pid), static_cast(batch.events[i+1].pid),
static_cast(batch.events[i+2].pid), static_cast(batch.events[i+3].pid), static_cast(batch.events[i+4].pid),
static_cast(batch.events[i+5].pid), static_cast(batch.events[i+6].pid), static_cast(batch.events[i+7].pid) }; __m256i
loaded_pids = _mm256_loadu_si256(reinterpret_cast(pids)); __m256i cmp_mask =
_mm256_cmpeq_epi32(loaded_pids, target_vec); int mask = _mm256_movemask_epi8(cmp_mask); for (size_t j = 0; j
< 8 && (i + j) < batch.count; ++j) { if (batch.events[i + j].pid == target_pid || target_pid == 0)
{ batch.events[write_idx++] = batch.events[i + j]; } } } batch.count = write_idx; return write_idx; } }; } // namespace
syspilot::pipeline ```
```

VOLUME 6: COLUMNAR TSDB STORAGE ENGINE & COMPRESSION MATH

6.1 Gorilla Floating-Point Encoding Engine Implementation

```
```cpp namespace syspilot::storage { class GorillaEncoder { private: uint64_t last_val_bits_{0}; uint32_t
last_leading_zeros_{0xFFFFFFFF}; uint32_t last_trailing_zeros_{0}; std::vector buffer_; public: void
encode_double(double value) { uint64_t val_bits; std::memcpy(&val_bits, &value, sizeof(double)); if (buffer_.empty())
{ append_bits(val_bits, 64); last_val_bits_ = val_bits; return; } uint64_t xor_val = val_bits ^ last_val_bits_; if (xor_val ==
0) { append_bit(0); } else { append_bit(1); uint32_t leading = __builtin_clzll(xor_val); uint32_t trailing =
__builtin_ctzll(xor_val); if (leading >= last_leading_zeros_ && trailing >= last_trailing_zeros_) { append_bit(0);
append_bits(xor_val >> last_trailing_zeros_, 64 - last_leading_zeros_ - last_trailing_zeros_); } else { append_bit(1);
last_leading_zeros_ = leading; last_trailing_zeros_ = trailing; append_bits(leading, 5); uint32_t length = 64 - leading -
trailing; append_bits(length, 6); append_bits(xor_val >> trailing, length); } } last_val_bits_ = val_bits; } private: void
append_bit(uint8_t bit) { /* Bitwise buffer write */ } void append_bits(uint64_t val, uint8_t num_bits) { /* Bitwise
buffer write */ } }; } // namespace syspilot::storage ```
```

# VOLUME 7: INVERTED INDEXING & VECTORIZED QUERY ENGINE

---

---

## 7.1 Roaring Bitmap Index Implementation

---

```
```cpp namespace syspilot::storage { class RoaringBitmapIndex { private: tsl::robin_map index_; public: void
insert(const std::string& tag, uint32_t row_id) { index_[tag].add(row_id); } roaring::Roaring query_and(const
std::string& tag1, const std::string& tag2) const { auto it1 = index_.find(tag1); auto it2 = index_.find(tag2); if (it1 ==
index_.end() || it2 == index_.end()) { return roaring::Roaring(); } roaring::Roaring result = it1->second; result &= it2-
>second; return result; } }; } // namespace syspilot::storage ```
```

VOLUME 8: MULTIMODAL CAUSAL REASONING

MULTIGRAPH

8.1 Causal Graph C++ Core Subsystem

```
```cpp namespace syspilot::causal { enum class NodeType : uint8_t { PROCESS, FILE, SOCKET, CGROUP, DEVICE };
enum class EdgeType : uint8_t { SPAWNED_BY, READS_FROM, WRITES_TO, BLOCKED_ON, CONTENTS_WITH };
struct GraphNode { std::string_view id; NodeType type; pid_t pid{0}; double cpu_pct{0.0}; double
read_rate_kb{0.0}; double write_rate_kb{0.0}; bool is_anomalous{false}; std::string_view anomaly_reason; }; struct
GraphEdge { std::string_view from_id; std::string_view to_id; EdgeType type; uint64_t latency_ns{0};
std::string_view details; }; class CausalGraph { private: tsl::robin_map nodes_; std::vector edges_;
memory::StringArena arena_; public: void add_node(GraphNode node) { node.id = arena_.allocate(node.id);
nodes_[node.id] = node; } void add_edge(GraphEdge edge) { edge.from_id = arena_.allocate(edge.from_id);
edge.to_id = arena_.allocate(edge.to_id); edges_.push_back(edge); } std::vector trace_root_cause(std::string_view
symptom_id) { std::vector path; tsl::robin_set visited; std::queue q; q.push(symptom_id);
visited.insert(symptom_id); while (!q.empty()) { auto curr = q.front(); q.pop(); path.push_back(curr); for (const auto&
edge : edges_) { if (edge.from_id == curr && visited.find(edge.to_id) == visited.end()) { visited.insert(edge.to_id);
q.push(edge.to_id); } } } return path; } }; // namespace syspilot::causal ```
```

# VOLUME 9: MULTI-TIER IPC INFRASTRUCTURE & SHARED MEMORY RINGS

---

---

## 9.1 Shared Memory Publisher Interface (`/dev/shm/syspilot\_metrics.ring`)

```
```cpp namespace syspilot::ipc { struct MetricsSnapshot { uint64_t timestamp_ns; double cpu_user_pct; double
cpu_system_pct; uint64_t memory_rss_bytes; uint64_t total_events_processed; uint64_t total_events_dropped; };
struct alignas(4096) SharedMetricsRegion { static constexpr size_t RING_SIZE = 1024; std::atomic write_head{0};
MetricsSnapshot snapshots[RING_SIZE]; }; class SharedMemoryPublisher { private: int fd_{-1};
SharedMetricsRegion* region_{nullptr}; public: bool init() { fd_ = shm_open("/syspilot_metrics.ring", O_CREAT |
O_RDWR, 0666); if (fd_ < 0) return false; ftruncate(fd_, sizeof(SharedMetricsRegion)); region_ =
static_cast( mmap(nullptr, sizeof(SharedMetricsRegion), PROT_READ | PROT_WRITE, MAP_SHARED, fd_, 0)); return
region_ != MAP_FAILED; } void publish(const MetricsSnapshot& snap) { if (!region_) return; uint64_t head = region_-
>write_head.load(std::memory_order_relaxed); region_->snapshots[head % SharedMetricsRegion::RING_SIZE] =
snap; region_->write_head.store(head + 1, std::memory_order_release); } }; } // namespace syspilot::ipc ```
```

VOLUME 10: ENTERPRISE PRODUCTION SAFETY

CONTROLS & RECOVERY PROTOCOLS

10.1 The 13 Production Safety Principles

1. **CPU Limit (< 1.0% Host Core):** Push-based eBPF tracepoints + epoll reactor pattern. 2. **Memory Bound (< 64MB RSS):** Pre-allocated slab bump arenas; zero dynamic heap growth. 3. **Zero Hot-Path Dynamic Allocations:** In-place ring buffer processing and arena strings. 4. **Lockless Multithreading:** Lock-free MPMC queue (`moodycamel::ConcurrentQueue`) and `tsl::robin_map`. 5. **Context Switch Minimization:** Adaptive wakeup high-watermark epoll polling (<100 wakeups/sec). 6. **Kernel-Userspace Responsibility Separation:** Raw event emission in eBPF; DWARF unwinding in userspace. 7. **Backpressure Handling:** Atomic event drop counters in kernel eBPF ring buffer headers. 8. **Event Loss Detection:** Monotonic sequence numbers per CPU stream. 9. **Adaptive Load Shedding:** Dynamic token bucket rate limiting at 80% host CPU utilization. 10. **Kernel Instability Immunity:** Static verification by Linux kernel eBPF verifier (zero panic risk). 11. **Process Fault Isolation:** Sandboxed daemon execution; host application non-interference. 12. **Vectorized Ingestion:** AVX2/AVX-512 SIMD pipeline supporting >5,000,000 events / sec / host. 13. **Resource Cgroup Enforcement:** Enforced systemd slice limits (`MemoryMax=128M`, `CPUQuota=5%`).